A · COMPRENDRE LE PROJET

Construire une solution BI avec l’IA

Python, ETL et entrepôts de données
Un projet Atlas, du premier calcul au rapport.

28 h

70 % pratique · 30 % théorie

Youssef Douma · Leaders University

Une réponse plausible peut être fausse

CANDIDAT FAUTIF CONSTRUIT

2 carnets à 4.500 TND
3 stylos à 1.500 TND

4 500 + 1 500 = 6 000

CE QUE NOUS AVONS VENDU

2 × 4 500 + 3 × 1 500
= 13 500 millimes

= 13.500 TND

Exemple pédagogique construit. Ce n’est pas une réponse observée d’une offre gratuite.

La preuve commence avant le code

Carnets

2 × 4 500 = 9 000

millimes

Stylos

3 × 1 500 = 4 500

millimes

13 500 millimes = 13.500 TND

Variation : 3 carnets et 3 stylos → 18 000 millimes.

Prédire le résultat, exécuter, expliquer l’écart. Le modèle propose ; vous vérifiez.

Votre résultat après 28 heures

1

Construire

Un pipeline Python, un entrepôt SQLite et un rapport HTML.

2

Vérifier

Les montants, les lignes rejetées et un ajout rejoué sans doublon.

3

Expliquer

Le grain, les unités, les clés et les limites de la décision.

Aucune mémorisation de syntaxe demandée. Vous pouvez consulter les repères.

Votre parcours : du projet à la preuve

A → B → C → D → E

Comprendre

Le cas Atlas et le résultat à construire.

Préparer

Les fichiers, les outils et le premier lancement.

Construire

Une fonction à la fois, avec l'IA et des tests.

Contrôler

Lire le rapport et vérifier ses limites.

Réaliser

Le défi CampusBox et votre preuve personnelle.

Les mêmes lettres A → E vous repèrent dans les guides, les captures et la présentation.

Atlas Papeterie prépare ses achats

1

Boutiques

NBE : Nabeul
TUN : Tunis

2

Produits

P01 Carnet · P02 Stylo
P03 Classeur
P04 Surligneur

3

Question métier

Quelles ventes observées faut-il examiner avant les prochains achats ?

Cas fictif. Les données ne décrivent ni stock, ni coût, ni marge, ni prévision.

Une seule chaîne, des preuves à chaque étape

1

Extraire

Lire les fichiers CSV sans changer leur sens.

2

Transformer

Normaliser, contrôler et calculer les montants.

3

Charger

Relier les tables et produire un rapport depuis SQLite.

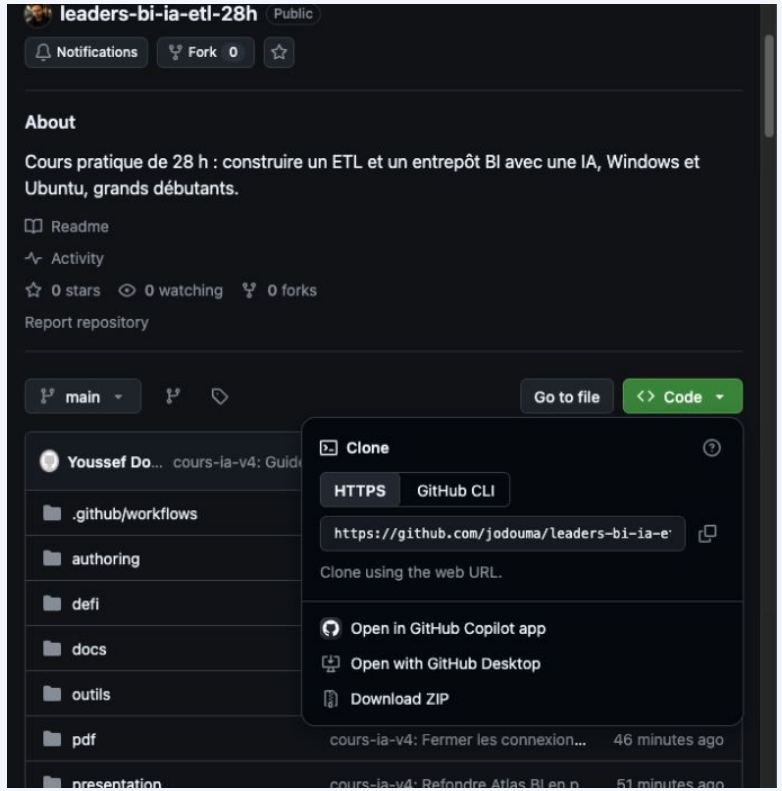
ETL = Extract, Transform, Load. Le rapport rend les résultats lisibles.

28 heures pour pratiquer et comprendre

1	**S01**	Installation + calcul	**3 h**
2	**S02–S03**	Lecture + qualité	**6 h**
3	**S04–S05**	Modèle + chargement	**6 h**
4	**S06–S07**	Incrément + rapport	**6 h**
5	**Défi**	CampusBox en autonomie	**7 h**

70 % pratique : 19 h 36 · 30 % théorie : 8 h 24. Installation incluse dans S01.

Télécharger le cours depuis le vrai dépôt



1. Ouvrir le dépôt du cours et le bouton Code.
2. Choisir Download ZIP ; ouvrir le fichier téléchargé.
3. Décompresser avant de modifier le dossier projet/.

Capture du projet GitHub, accès public.

Vrai dépôt Atlas — capture du navigateur réalisée le 27 octobre 2026. Aucun compte étudiant affiché.

Éditeur, terminal, navigateur : trois espaces

E

ÉDITEUR

`calcul.py`

J’écris et j’enregistre.

T

TERMINAL

`python calcul.py`

Je lance une commande.

N

NAVIGATEUR

`rapport_atlas.html`

Je lis les résultats.

Schéma de principe : l’éditeur modifie les fichiers ; le terminal exécute ; le navigateur affiche le rapport.

Windows : préparer une fois, puis travailler

1

Vérifier

Windows 11 pris en charge et à jour, virtualisation active et WSL 2 opérationnel selon le guide officiel.

2

Installer

Docker Desktop : choisir les conteneurs Linux. Ouvrir l’éditeur et le terminal du dossier décompressé.

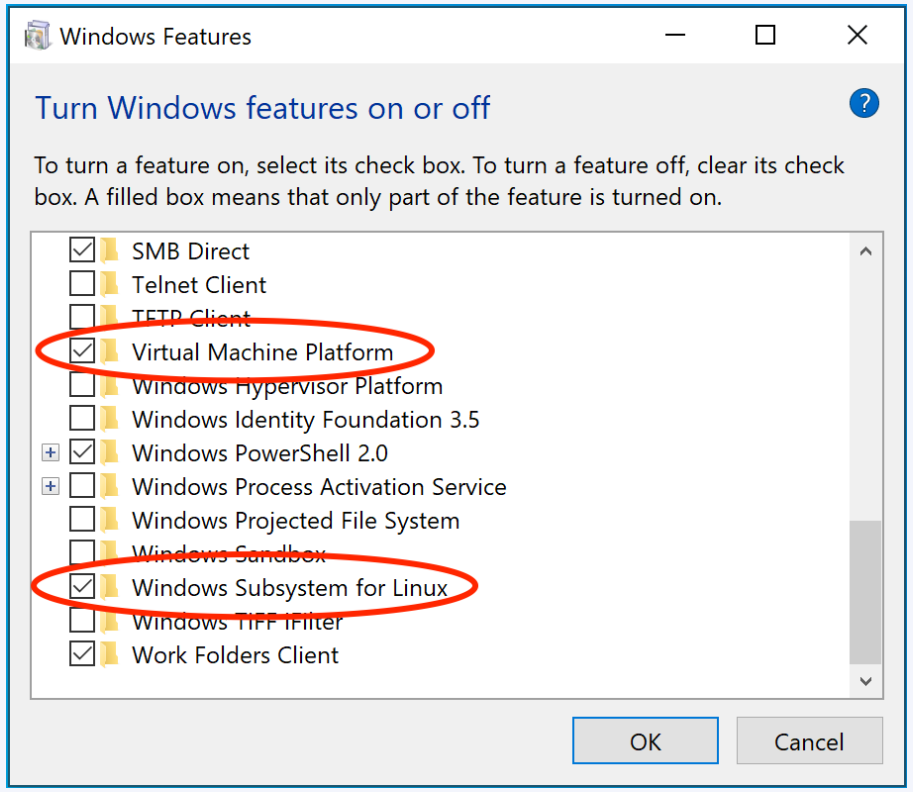
3

Confirmer

Docker Desktop démarré. Dans le terminal :
docker --version
docker compose version

Docker Desktop est propriétaire ; usage éducatif gratuit selon ses conditions. Voir le PDF 01.

Windows : reconnaître les deux fonctions utiles



1. Virtual Machine Platform : prise en charge de la virtualisation.
2. Windows Subsystem for Linux : la couche WSL utilisée par Docker Desktop.
3. Sur Windows 11, suivre le parcours actuel du PDF 01. L'apparence peut différer.

Capture officielle,
non réalisée sur notre poste.

Windows Features : interface historique en anglais. Documentation Docker, licence Apache 2.0.

Ubuntu ou installation Docker bloquée

1

Ubuntu

Installer Docker Engine et le plugin Compose depuis le dépôt officiel. Vérifier les deux commandes de version.

2

Repli natif

Installer Python 3.13. Dans projet/ : python3 verifier.py 01. Sous Windows : py -3.13 verifier.py 01.

3

Même projet

La bibliothèque standard suffit. Aucun serveur cloud ni clé API requis pour exécuter le projet.

Ubuntu : Docker Engine est open source. Les droits Docker donnent des capacités importantes sur la machine.

Docker : un modèle et son exécution

1

IMAGE

Modèle prêt à lancer

Python 3.13 + environnement

2

CONTENEUR

Exécution de cette image

Un programme qui tourne

3

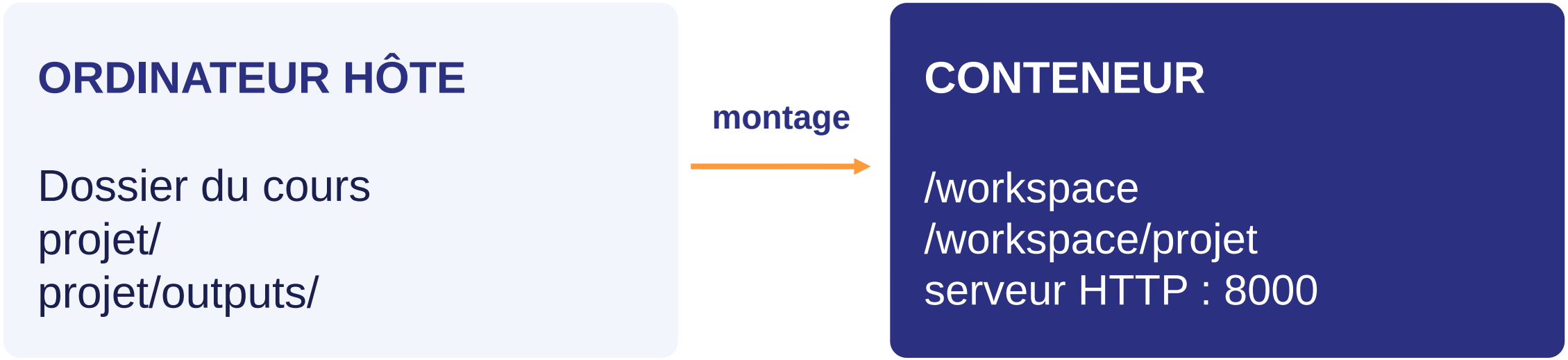
VOS FICHIERS

Dossier du cours sur l’hôte

Code, données et sorties

Dockerfile est la recette ; l’image contient Python ; le conteneur exécute. Vos fichiers restent dans le dossier du cours.

Le dossier partagé et localhost



Navigateur → localhost:8000 → port 8000 du conteneur

Le montage relie vos fichiers au conteneur. Le port publié sur 127.0.0.1 est accessible depuis votre ordinateur.

Premier lancement : dans le dossier du cours

```
docker compose build

docker compose run --rm atelier python calcul.py
```

1. La construction télécharge Python une première fois.
2. Exécuter la seconde commande sur une ligne.
3. Enregistrer calcul.py avant de relancer.

Une commande → un résultat local. --rm supprime le conteneur terminé, pas le dossier partagé.

L’IA web suffit pour suivre le cours

1

Socle gratuit

Un chat web accessible avec votre compte. Ses limites varient ; fractionner les demandes et reprendre plus tard.

2

Options payantes

Un abonnement peut offrir plus de capacité. Aucune fonction payante n’est exigée par les missions.

3

CLI facultative

Un assistant en terminal peut agir sur les fichiers. Son accès dépend de votre compte et de l’offre en vigueur.

Ne jamais déduire « chat gratuit » ⟹ « CLI gratuite ». Vérifier l’éligibilité avant toute installation.

Une boucle courte économise votre attention

1	2	3	4	5
PLAN	**PROMPT**	**CODE**	**TEST**	**DIFF**
But + contrat	Petit contexte	Une fonction	Résultat local	Lire et garder

Si le test échoue : joindre l'erreur utile et reprendre au prompt.

Cible de travail : 2 à 4 demandes par mission, pas un quota garanti. Un échec local apporte le contexte suivant.

Un prompt utile tient dans un petit périmètre

```
But : calculer un total en millimes.
Fichier : projet/calcul.py.
Entrée : quantité et prix entiers.
Sortie : un entier.
Propose seulement calculer_total.
Explique le test 13 500 puis 18 000.
```

1. Donner le contrat avant le code.
2. Joindre seulement la fonction et l’erreur utile.
3. Relancer localement et lire la modification.

Pour le chat web : copier une petite fonction. Pour une CLI : limiter son périmètre au projet.

Gardez la main sur les changements

1

Avant

Sauvegarder un jalon. Préciser le fichier autorisé et le comportement attendu.

2

Pendant

Lire les demandes de permission. Accepter seulement une action comprise et utile.

3

Après

Comparer avant / après (diff), lancer le contrôle local et expliquer une ligne modifiée.

Ne pas contourner les permissions, transmettre des secrets, ni modifier les contrôles pour les faire passer.

Jalon de départ : votre poste répond

00 Ouvrir, enregistrer, exécuter.

1. Montrer le dossier projet/ et distinguer calcul.py de outputs/.
2. Exécuter le fichier de départ et lire le message affiché.
3. Conserver une trace de la commande, puis décrire son rôle.

Si Docker bloque : suivre le repli Python du PDF 01. L’objectif est un premier résultat local.

S01 · Un entier qui représente de l’argent

Carnets

2 × 4 500 = 9 000

millimes

Stylos

3 × 1 500 = 4 500

millimes

13 500 millimes, stockés comme un entier

1 TND = 1 000 millimes. Affichage : 13.500 TND.

quantity = nombre d’unités ; unit_price_millimes = prix d’une unité ; amount = quantité × prix.

Une fonction a une entrée et une sortie

```
def calculer_total(ventes):
    total = 0
    for vente in ventes:
        total += (vente["quantity"]
          * vente["unit_price_millimes"])
    return total
```

1. ventes est une liste de lignes.
2. Chaque ligne porte deux valeurs entières.
3. return donne le résultat au programme appelant.

Contrat : calculer_total(ventes) → int. Le code se lit avant de se modifier.

S01 · Corriger puis faire varier

01 Obtenir un résultat dont vous connaissez la valeur.

1. Faire proposer la fonction ; lire chaque opération.
2. Vérifier 13 500, puis prévoir et tester 3 carnets : 18 000.
3. Restaurer le cas initial et exécuter verifier.py 01.

Preuve : deux valeurs + une phrase expliquant pourquoi le prix seul ne suffit pas.

S01 · À ce stade, je sais…

1

Lire

Une quantité, un prix unitaire, un total et une unité.

2

Exécuter

Enregistrer un fichier puis lancer la bonne commande.

3

Vérifier

Prédire une variation et expliquer un résultat local.

Une explication + une exécution + une trace conservée.

S02 · Que représente une ligne ?

UNE LIGNE = UNE VENTE ÉLÉMENTAIRE

QUOI ?	OÙ ?	QUAND ?	COMBIEN ?
P01 · Carnet	**NBE · Nabeul**	**05/01/2026**	**2 unités**

ID S001 : stable même quand le fichier est rejoué.

Grain Atlas : une vente élémentaire d’un produit, dans une boutique, à une date. sale_id identifie cette vente.

Un CSV devient une liste de dictionnaires

Colonne CSV	Valeur lue	Sens
sale_id	"S001"	Vente stable
product_id	"P01"	Produit vendu
quantity	"2"	Texte à convertir
unit_price_tnd	"4.500"	Prix d’une unité

CSV = texte tabulaire. DictReader associe le nom de colonne à sa valeur ; rien n’est encore validé.

Lire depuis le dossier du fichier Python

```
from pathlib import Path
import csv
ROOT = Path(__file__).resolve().parent

with chemin.open(encoding="utf-8",
                 newline="") as fichier:
    lignes = list(csv.DictReader(fichier))
```

1. Écrire lire_csv(chemin) dans lecture.py.
2. chemin désigne un fichier ; ROOT désigne projet/.
3. Lire ne nettoie pas et ne calcule pas encore.

Extrait illustratif dans la fonction lire_csv. ROOT / "data" / "ventes_initiales.csv" construit le chemin.

S02 · À ce stade, je sais lire les données

02

Faire entrer les 16 lignes, sans en perdre une.

1. Proposer et tester lire_csv(chemin), puis verifier.py 02.
2. Afficher le nombre de lignes et les clés de la première.
3. Expliquer le grain et pourquoi "2" est encore du texte.

Preuve : 16 lignes lues. Si fichier introuvable, inspecter le chemin et la décompression.

S03 · La qualité préserve le sens

1

Normaliser

" nbe " devient "NBE". Même boutique, écriture cohérente.

2

Valider

Quantité > 0, prix > 0, date réelle et références connues.

3

Tracer

Conserver le motif du rejet et identifier les doublons.

Ne jamais inventer une date ou un produit pour conserver une ligne incorrecte.

Rejet, doublon et conflit : trois décisions

Situation	Décision	Pourquoi
Quantité 0 / P99 / 30 février	Rejeter + motif	Vente non exploitable
Même ID, même contenu	Compter un doublon	Même vente répétée
Même ID, contenu différent	Arrêter : conflit	Ne pas choisir en silence

Le nettoyage conserve les six champs source normalisés et ajoute les deux montants entiers.

Lire un prix sans perdre les millimes

CSV : texte

"4.500"

TND

→

Lecture exacte

Decimal("4.500")

× 1 000

→

Stockage

4 500

millimes · entier

Decimal lit le texte décimal. Vérifier la précision et la positivité, puis stocker un entier de millimes.

Aucune ligne ne disparaît sans explication

16
lignes brutes

12
retenues

1
doublon

3
rejets

16 = 12 + 1 + 3

Base Atlas : 32 unités vendues et 99 000 millimes retenus. Les comptes de qualité expliquent l’écart.

S03 · Transformer avec des traces

03 Écrire nettoyer(lignes, produits, boutiques).

1. Demander une petite fonction à la fois : normaliser puis contrôler.
2. Tester les erreurs fournies et le doublon, puis verifier.py 03.
3. Présenter retenues, rejets et doublons avec un exemple.

Preuve : 12 / 3 / 1 et 99 000 millimes. Un conflit de contenu doit provoquer une erreur claire.

S03 · À ce stade, je sais…

1

Réparer l’écriture

Une normalisation conserve la signification.

2

Préserver les faits

Un rejet a un motif ; un doublon n’est pas une nouvelle vente.

3

Contrôler le calcul

Les millimes restent des entiers du calcul au stockage.

Une explication + une exécution + une trace conservée.

S04 · Organiser pour poser des questions

1

Table

Des lignes de même nature et des colonnes nommées.

2

Clé primaire

Une valeur unique désigne une ligne. Exemple : sale_id.

3

Clé étrangère

Une valeur relie une vente à une ligne de dimension.

SQLite stocke l’entrepôt dans un fichier local. SQL permet de le créer et de l’interroger.

Une étoile autour des faits de vente



Le fait répond « combien ? ». Les dimensions répondent « quoi, où, quand ? ». Chaque flèche se lit 1 vers N.

Identifiant métier et clé de l’entrepôt

Catalogue métier	Dimension	Fait de vente
P01 = Carnet	product_key = 1	S001 → product_key 1
P02 = Stylo	product_key = 2	S002 → product_key 2
Code du produit	Clé locale de liaison	Référence la dimension

product_id est le code source. product_key est la clé locale. Une vente conserve les références vers les dimensions.

Décrire des règles dans le schéma SQL

```
sale_id TEXT PRIMARY KEY,
product_key INTEGER NOT NULL
  REFERENCES dim_product(product_key),
quantity INTEGER CHECK (quantity > 0),
unit_price_millimes INTEGER
  CHECK (unit_price_millimes > 0),
amount_millimes INTEGER
  CHECK (amount_millimes =
    quantity * unit_price_millimes)
```

1. Extrait de CREATE TABLE fact_sales.
2. Activer PRAGMA foreign_keys = ON.
3. Le schéma complète les contrôles Python.

modele.py : creer_modele(chemin_base) crée les tables si absentes, sans effacer les données.

S04 · À ce stade, je sais relier les tables

04 Créer trois dimensions et une table de faits.

1. Faire dessiner puis expliquer l’étoile avant de proposer le SQL.
2. Créer le fichier SQLite, puis exécuter verifier.py 04.
3. Montrer une clé primaire et une clé étrangère sur le dessin.

Preuve : dim_product, dim_shop, dim_date et fact_sales. Une seconde création ne doit rien effacer.

S05 · Charger dans le bon ordre

1

Dimensions

Insérer les produits, boutiques et dates. Retrouver leurs clés.

2

Faits

Insérer chaque vente avec ses références et ses mesures.

3

Contrôle

Compter les faits et additionner les montants depuis SQLite.

chargement.py : charger(ventes, produits, boutiques, chemin_base) → inserted / existing.

Une transaction protège tout le lot

DÉBUT

Ouvrir une transaction

→

LOT

Vérifier puis insérer

→

FIN

Tout valider ensemble

Conflit détecté → rollback : aucune insertion de ce lot.

Même ID + même contenu : existing. Même ID + autre contenu : erreur et annulation de tout le lot.

SQL répond à la question métier

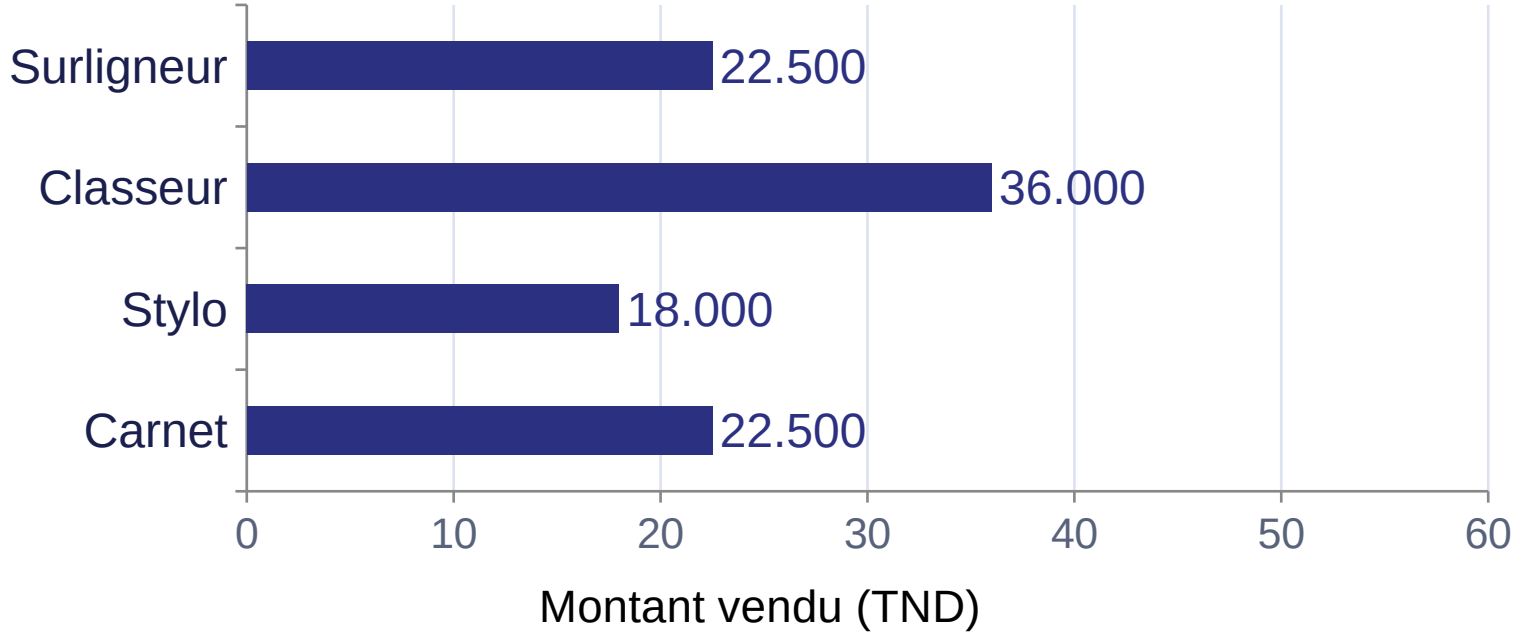
```
SELECT p.product_name,
       SUM(f.amount_millimes) AS total
FROM fact_sales AS f
JOIN dim_product AS p
  ON p.product_key = f.product_key
GROUP BY p.product_key, p.product_name
ORDER BY total DESC;
```

1. JOIN retrouve le nom du produit.
2. GROUP BY rassemble les ventes du même produit.
3. SUM additionne des montants, pas des prix.

Une jointure 1 vers N correcte ne multiplie pas une vente. Vérifier aussi COUNT(*) avant / après jointure.

Base Atlas : ventes par produit

Du 05 au 07 janvier 2026 · 12 faits · total 99.000 TND



Classeur 36.000 TND

Premier montant vendu sur la période.

Chaque barre représente un montant vendu en TND. Ces ventes ne donnent pas la marge.

S05 · Recharger la base sans doubler les ventes

05

Charger, contrôler, puis relancer le même lot.

1. Tester charger et lancer verifier.py 05.
2. Premier passage : inserted = 12 ; existing = 0.
3. Deuxième passage : inserted = 0 ; existing = 12.

Preuve : le total reste 99 000 millimes et la base garde 12 faits. Lire aussi le test du conflit.

S05 · À ce stade, je sais…

1

Stocker

Les dimensions décrivent ; les faits mesurent.

2

Interroger

Relier, regrouper et sommer avec SQL.

3

Protéger

Un lot est cohérent et un rejeu identique ne crée rien.

Une explication + une exécution + une trace conservée.

S06 · Un petit ajout, une mise à jour fiable

Vente	Contenu	Effet
S013	1 carnet · 4.500 TND	+1 fait
S014	2 classeurs · 12.000 TND	+1 fait
S012	Déjà présente, identique	0 fait

2 faits nouveaux · +3 unités · +16.500 TND

ventes_ajout.csv : S013 et S014 sont nouvelles ; S012 existe déjà. Ajouter = nettoyer puis charger, sans vider la base.

Le second passage ne change plus rien

Avant ajout

12 faits

99.000 TND

Premier passage

14 faits

115.500 TND

Deuxième passage

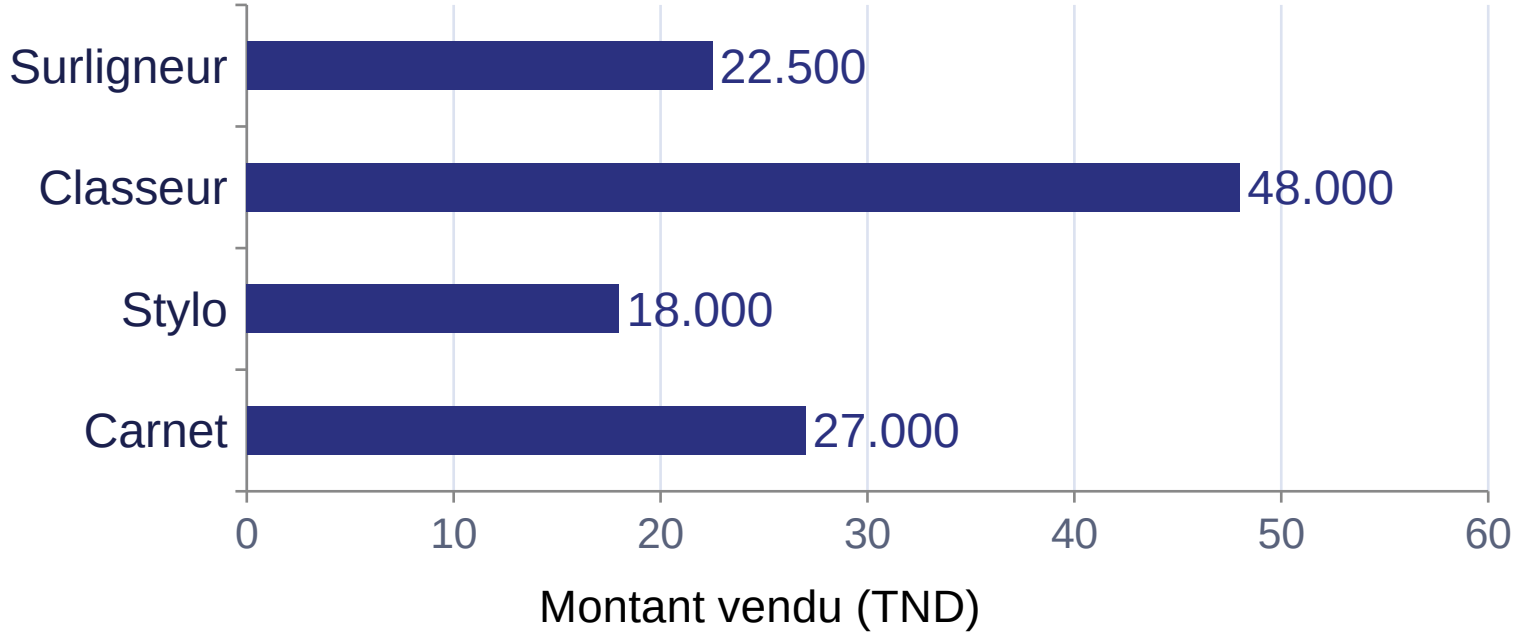
14 faits

115.500 TND

Premier ajout : 2 nouvelles + 1 existante. Rejeu du même fichier : 0 nouvelle + 3 existantes.

Après ajout : annoncer la nouvelle période

Du 05 au 08 janvier 2026 · 14 faits · total 115.500 TND



+16.500 TND

Deux ventes nouvelles ; un jour de plus.

99.000 sur 3 jours et 115.500 sur 4 jours : cette hausse ne prouve pas une amélioration de performance.

S06 · À ce stade, je sais mettre à jour

06 Réutiliser les fonctions déjà fiables.

1. Écrire ajouter(source, base, data) dans incremental.py.
2. Lancer verifier.py 06 et rejouer le même ajout.
3. Expliquer 12 → 14 → 14 faits et 32 → 35 unités.

Preuve : 115 500 millimes à chaque rejeu. Si le total augmente encore, inspecter sale_id.

S07 · Un rapport est une preuve lisible

1

Contexte

Le nom du cas, les dates et le grain apparaissent dans le rapport.

2

Résultats

Les indicateurs et barres viennent de la base réelle, avec leurs unités.

3

Interprétation

Une observation, une action prudente et une limite explicite.

rapport.py : generer(chemin_base, chemin_html) → résumé + HTML autonome.

Distinguer observation et décision

CE QUE LES DONNÉES DISENT

Le classeur représente
48.000 TND vendus
du 05 au 08 janvier.

14 faits au total.

CE QU'IL RESTE À VÉRIFIER

**Examiner les stocks
et les coûts avant achat.**

**Aucune conclusion de marge
ou de prévision.**

Un rapport utile aide à poser la prochaine question ; il ne fabrique pas la réponse manquante.

Exécuter toute la chaîne et ouvrir le rapport

```
docker compose run --rm atelier python executer.py
docker compose up -d

http://localhost:8000/rapport_atlas.html

docker compose down
```

1. Exécuter la chaîne avec la première commande.
2. Ouvrir l’adresse dans le navigateur de l’hôte.
3. Après vérification, arrêter le serveur avec down.

Depuis le dossier du cours. Repli natif : voir le PDF 01 pour lancer le serveur de outputs/.

Votre cible : le vrai rapport Atlas dans le navigateur



Rapport réel : Docker Linux / Python 3.13 sur hôte macOS ARM. Navigateur du poste auteur, 27 octobre 2026.

1. Lire les dates avant de comparer.
2. Retrouver 14 faits, 35 unités et 115.500 TND.
3. Défiler : produits et limites métier.

Capture du projet
Données fictives Atlas.

Quand cela bloque : réduire le problème

1

Lire

Dernière ligne de l’erreur, fichier et numéro de ligne. Reproduire une seule commande.

2

Isoler

Demander une hypothèse sur la petite fonction concernée. Joindre entrée, attendu et obtenu.

3

Réparer

Inspecter la différence, relancer le test concerné, puis le parcours complet.

Ne pas réécrire tout le projet à chaque erreur. Les contrôles sont des repères à comprendre.

S07 · À ce stade, je sais livrer une chaîne

07

Faire fonctionner le rapport depuis une base réelle.

1. Tester rapport.py, executer.py puis verifier.py 07 et tout.
2. Ouvrir le HTML et vérifier période, total et barres.
3. Relancer executer.py et démontrer un résultat stable.

Preuve : 14 faits, 35 unités, 115.500 TND + un rapport lisible et une limite métier.

E · RÉALISER LE DÉFI

Défi CampusBox
Votre transfert en 7 h

Même démarche, nouvelle activité :
fiabiliser des ventes de repas sur le campus.

7 h

Équipe de 3 au maximum · preuves individuelles

Youssef Douma · Leaders University

CampusBox : choisir le périmètre

1

Source

Une ligne décrit une vente de portions. Garder l’identifiant de vente stable.

2

Statuts

Seules les ventes SERVIE alimentent les indicateurs.

3

Trace

ANNULEE et EN_ATTENTE restent expliquées dans le journal hors périmètre.

Ne pas traiter une commande en attente comme un repas vendu. Consulter defi/ et le PDF 04.

CampusBox : quelles preuves faut-il produire ?

1

Base

Réconcilier lignes lues, retenues, doublons et motifs hors périmètre.

2

Ajout

Distinguer les ventes nouvelles des identifiants déjà présents.

3

Rejeu

Relancer le même ajout et prouver que les indicateurs restent stables.

Conserver vos résultats et expliquer leur sens. Les réponses numériques sont réservées au corrigé.

Au plus trois personnes, une preuve par personne

1

Piloter

Formuler le contrat, prévoir les valeurs et garder les jalons.

2

Construire

Proposer une petite modification et lire le diff avec le groupe.

3

Vérifier

Exécuter, inspecter les sorties et expliquer les limites.

Faire tourner les rôles. Seul : les exercer successivement. Chacun montre une modification et une explication.

Sept heures avec des jalons visibles

1	0 h–1 h	Comprendre données et périmètre	**Contrat écrit**
2	1 h–3 h	Lire, nettoyer, calculer	**Base expliquée**
3	3 h–5 h	Modéliser et charger	**SQL + contrôles**
4	5 h–6 h	Ajouter et rejouer	**Stabilité prouvée**
5	6 h–7 h	Rapport et démonstration	**Livraison expliquée**

Cible de travail, à adapter au rythme réel. Garder un résultat utilisable à chaque jalon.

CampusBox · Tester une variation comprise

DÉFI

Prouver votre raisonnement, pas seulement une sortie.

1. Chaque personne prédit l’effet d’une petite variation.
2. Modifier sur une copie, exécuter et expliquer le résultat.
3. Restaurer la source et conserver commande, preuve et diff.

Une erreur expliquée et réparée apporte une preuve d’apprentissage. Les repères restent autorisés.

Livrer un projet que quelqu’un peut vérifier

1

Fichiers

Code exécutable, sources et rapport. README court : démarrer et reproduire.

2

Preuves

Comptes qualité, total de base, ajout, rejeu stable et résultat SQL.

3

Explication

Le grain, les unités, une décision prudente, une limite et une contribution personnelle.

Avant de déposer : refaire le démarrage depuis une copie propre et vérifier les fichiers réellement inclus.

À ce stade, je sais construire une solution BI

1

Données fiables

Lire et conserver le sens, contrôler les rejets et les montants.

2

Entrepôt utile

Relier les tables, interroger et recharger sans doublon.

3

IA maîtrisée

Définir un petit contrat, vérifier localement et expliquer le changement.

Une explication + une exécution + une trace conservée.

E · DU FICHIER À LA DÉCISION

Une proposition devient utile quand vous pouvez la vérifier.

1. Un contrat court avant chaque demande.
2. Un test local avant chaque confiance.
3. Une explication claire avant chaque décision.

Prochaine étape : réutiliser la démarche sur des données dont vous comprenez le sens.